

빠른 가시성 제외, 레이 트레이싱, 범위 검색을 위한 구 트리

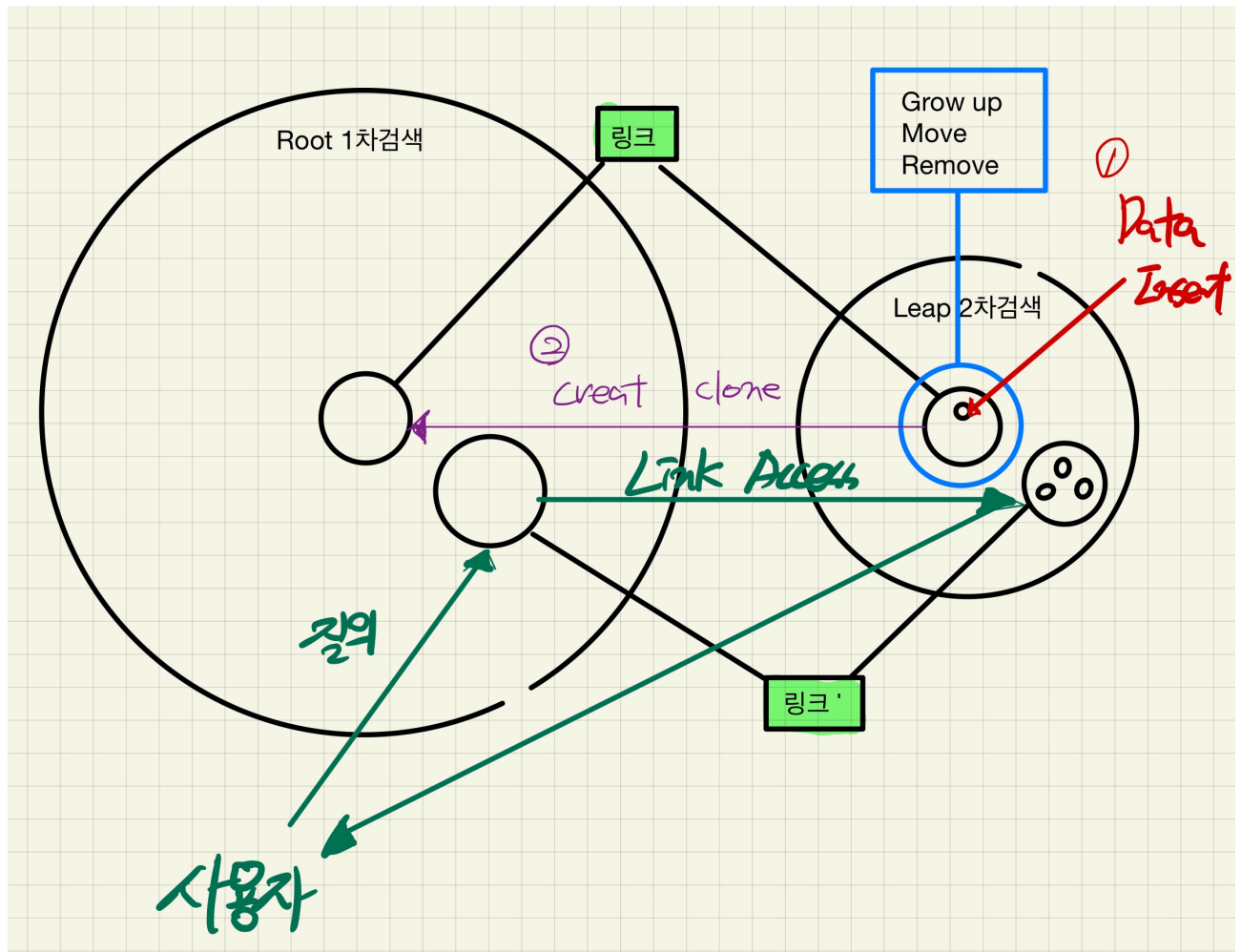
캠프 설치 불가 지역을 빠르게 검색하기 위한 수단으로 적용할 예정

<https://gpgstudy.com/catalog/gpg2>

<https://gpgstudy.com/catalog/2:4.3>

해당 글은 적용된 알고리즘을 공유하기 위한 내용 및 설계 내용을 포함합니다.

- 알고리즘 해석
 - 2개의 구트리 노드 존재
 - Root 노드
 - 검색을 위해 존재하는 노드
 - Leaf 노드
 - 데이터 입력을 위해 존재하는 노드
 - 동작설명
 - 최초 생성되는 노드(Root, Leap)는 SPF_ROOTNODE 플레그로 범위가 재계산되거나 병합되지 않는다.
 - 외부에서 입력되는 Sphere 정보는 Leap노드에서 관리된다.
 - 최초로 입력시 Leap노드에는 SuperNode(중간노드) 가 없는 상태임으로 SuperNode가 신규 생성되는데 이때 Root노드에도 동일한 정보로 노드가 생성되며 상호 LinkData(서로의 포인터)로 연결된다.
 - 중간입력시 이미 이전에 생성된 SuperNode가 있기때문에 포함가능 하다면 자식으로 설정
 - 포함 불가능하다면 한번 증가가능한 폭내에 존재하는지 재검사
 - 가능하면 자식,
 - 불가능하면 신규 SuperNode 생성
 - 퍼포먼스를 얻는방법
 - Root노드는 관리하는 노드정보가 SuperNode
 - 좀더 적은 Iteration으로 전체범위에서 대상을 찾아냄
 - 이후 링크된 Leap노드를 재검사하여 데이터 입력 노드를 찾는 구조
 - 트리구조에 상위 구는 하위구를 모조건 포함하는 관계임으로 부모자식, 이웃노드를 재귀호출 하면서 Iteration 비율을 줄임
 - 지정된 갯수만큼 미리 생성하는 고정풀을 사용하여 Runtime 메모리 할당 퍼포먼스를 줄임
 - 사용된 Tech
 - 지정된 갯수만큼 미리 생성하는 고정풀
 - 선입선출 큐
 - 선, 구 Intersect
 - 구, 구 Intersect
 - 이미지 설명



- 작업목록

- sample code 최신화 :
 - 문법 및 컴파일 오류수정
 - 언리얼 객체 활용
 - 언리얼 Intersect 로직 활용
 - Smart Pointer 적용 → 부분 적용(메모리풀을 new 를 활용하지 않는 객체풀 변경)
- 검증로직 작성
 - 로직 적용 대상에 인스턴스 생성 (GsGameObjectManager)
 - 맵별 데이터 입력 처리
 - Range Test 로직 적용
 - Visualize 처리(확인용)
- 퍼포먼스 체크
 - 시간 검증

- 클래스 다이어그램

- 시퀀스 다이어그램

- 검증로직 작성

